Outage-Aware Loss Handling and Energy-Efficient Reporting for River Water-Quality Telemetry over LPWANs

Kofi Sarpong Adu-Manu and Stephen Kwaku Azongo

***Abstract*—Battery-powered river water-quality telemetry suffers short radio losses and long delivery gaps, yet recovery strategies are usually evaluated under independent packet-loss assumptions. This study characterised losses in four operational records (two water-quality deployments and two LoRaWAN networks) and evaluated retransmission, buffering, and reconstruction with trace-driven energy models for LoRaWAN (long-range wide-area network) and narrowband Internet of Things (NB-IoT). Gaps of at least 24 h held 50.7% and 82.9% of the missing readings in the two water-quality records, and a geometric gap-length model was rejected (bootstrap test, $p \leq 0.005$). Automatic repeat request (ARQ) recovered less than one-fifth of the missing readings, even under optimistic retry timing, at 2.3×–7.3× the energy of unconfirmed LoRaWAN reporting, whereas buffering with back-off recovered all generated readings. Neither simple nor deep reconstruction kept most 24-h gaps within tolerance. An outage-aware scheme (hourly batching, immediate alarm uploads, a bounded-error trigger, and buffered backfill) was tested with frozen rules on held-out 2024 data from 33 river sites. With the link up, it matched per-reading reporting in alarm latency for detected events and in the share of server estimates within tolerance (-0.01 to $+0.56$ percentage points), while per-reading ARQ with buffering used 1.34×–2.53× the transmission energy and 1.15×–1.41× the node energy. These findings support outage-aware buffering and selective reporting.**

***Index Terms*—LoRaWAN, NB-IoT, packet loss, outage recovery, energy efficiency, event-driven reporting.**

I. Introduction

RIVER and reservoir monitoring using in situ sensor nodes must work where field visits are expensive and power is scarce [1], [2]. A typical node samples temperature (T), specific conductance (EC), dissolved oxygen (DO), and pH every 5–30 min and sends the readings over a cellular or low-power wide-area network (LPWAN). Such nodes lose data. The InWaterSense deployment on the Sitnica river in Kosovo missed 46.0% of its expected readings [3]; the CSIRO water-quality network in Australia averaged approximately 64% delivery because the gateway's connection to the server "was down occasionally" [4]; and deployment reports describe gateways that lose power every night [5], sinks that stop reconnecting until rebooted [6], [7], outages of days to weeks in a subarctic DASH7 network [8], and prolonged cellular outages together with a database that was offline for weeks [9].

A node can retransmit a lost reading until it is acknowledged (automatic repeat request, ARQ), buffer it and backfill it later, or leave the server to reconstruct it from the site's history or its neighbours. The correct response depends on how data are lost: retransmission helps against a short radio fade but wastes energy against a gateway that is down for a day, and reconstruction can fill ten minutes but not a week. Yet, the analyses behind these remedies assume short losses: estimation over lossy links uses independent or short-memory (Markov fading) packet errors [10]–[12], and imputation methods are evaluated on random masks or simulated failures of a few hours [13]–[16]. Deployment reports describe long outages, but how long real delivery gaps last, how much of the missing data they hold, and what this implies for the energy and timeliness of each remedy have not been measured across records and compared remedy by remedy.

The contribution of this study is not a new retransmission or prediction primitive; ARQ, buffering, back-off, batching, and error-bounded (dual-prediction) reporting are established. It is an evidence-to-design chain for outage-aware, energy-efficient telemetry. Real delivery records measure how long losses last and where the missing data are. The records are replayed through a measurement-parameterised LPWAN energy model, with explicit bounds on what they cannot show (why a reading is missing, and when within a slot the network returned), to decide when each remedy pays. The resulting reporting design is fixed on one year of river data and tested on a held-out year. Four research questions structure this study:

- RQ1 In the records studied, how long are the delivery gaps, how much of the missing data do long gaps hold, and is a geometric (memoryless) gap-length model adequate?
- RQ2 Under these gaps, what do retransmission, buffering with back-off, and batching recover, at what energy and delay, on LoRaWAN and NB-IoT?
- RQ3 Can reconstruction from a site's own history or its neighbours replace delivery during a gap?
- RQ4 Can one reporting scheme raise alarms as fast as per-reading reporting and keep the server's real-time picture within a stated tolerance as often, at substantially less energy than per-reading retransmission, on data not used for its design?

The contributions are as follows:

- Empirical loss structure (Section IV): In four records, losses fall into short radio losses and rare long delivery gaps, which hold most of the missing readings in the water-quality records; a geometric gap-length model is strongly unsupported.

K. S. Adu-Manu and S. K. Azongo are with the Department of Computer Science, University of Ghana, Accra, Ghana. Corresponding author: Kofi Sarpong Adu-Manu (ksadu-manu@ug.edu.gh).

- Consequence analysis with explicit bounds (Sections V and VI): ARQ recovers under a fifth of the missing readings even under optimistic retry timing, at 2.3×–7.3× the energy of unconfirmed reporting on LoRaWAN, whereas buffering with back-off recovers every generated reading at an energy -14.9 to $+25.9\%$ different from that of recovering nothing on LoRaWAN, a difference whose sign depends on the radio's confirmed-to-unconfirmed energy ratio. Of the evaluated reconstruction estimators, including two deep imputers, the best for each gap keeps only 31% of day-long gaps within tolerance (95% CI 18%–49%).
- An outage-aware reporting design, confirmed on held-out data (Section VII): Bounded event-driven batching combines hourly batches, immediate alarm uploads, a bounded-error trigger that mirrors the server's estimate, and buffering with back-off. It was developed at 30 USGS river sites in 2023 and tested, with its rules frozen, at 33 sites in 2024.
- Limits of further protocol work (Section VIII): A probing schedule fitted to the records' own gap lengths saves at most 2.9% of the design's energy on LoRaWAN, and once readings are batched, transmission is only 16%–54% of the node's energy; simple rules therefore leave little for a more elaborate reporting protocol to gain under these gaps.

Scope: The two water-quality records cannot fully separate a network outage from a node that stopped producing readings; therefore, this study speaks of delivery gaps and treats buffering's recovery as an upper bound. Energy comes from a model parameterised by published measurements, not from hardware built for this study, and the claims are ratios between strategies on the same radio.

Section II reviews related work. Section III describes the data and models. Sections IV–VII answer RQ1–RQ4, and Section VIII discusses guidance and limitations.

II. Related Work

How sensor networks lose data: Packet delivery in dense sensor networks varies widely between links and over time [17]. Kong et al. [13] analysed three real datasets with losses of 23–64% and identified five loss patterns, from random loss to "successive loss in a row" when a node fails, without measuring durations. Deployment reports describe long gaps anecdotally: a gateway losing its server connection [4], nightly gateway power loss [5], sinks and modems that stop until reset [6], [7], and outages of days to weeks in a subarctic DASH7 network [8]. Public LoRaWAN datasets now allow loss to be measured from frame counters: LoED [18] and a 16-month deployment at the University of Virginia (UVA) [19].

Estimation and retransmission: Kalman filtering with intermittent observations was analysed under independent arrivals [10]; remote estimation asks whether to retransmit a failed packet or send a fresh one under independent or Markov fading channels [11], [12], and the age of information formalises timeliness [20], [21]. Application-layer coding recovers LoRaWAN frames lost to the radio [22]. These analyses model losses that last a few packets; none of them models periods of days with no communication.

Confirmed traffic and bursts in LoRaWAN: Because the gateway is half-duplex and its downlink is duty-cycle limited, acknowledgements become scarce as a cell grows, and confirmed traffic sharply lowers the performance of large deployments [23], [24]. Group acknowledgements [25] and scheduled bulk transmission of buffered data [26] reduce this cost, Burst-MAC organises nodes into time-slotted groups when an event triggers a burst of traffic [27], and CADD adapts the duty cycle and data reduction jointly to control congestion [28]. These protocols are evaluated on a network that stays reachable; this study takes their conclusion about acknowledgements as given and asks what a backlog left by a measured outage adds to it.

Reconstruction: Compressive and low-rank gathering [29]–[31], and deep spatio-temporal imputation with graph networks [14], [15], [32], [33] or self-attention [34], [35], exploit spatial and temporal correlation. Their evaluations use random losses [16] or simulated blocks of 1–4 h [14], [15], without a per-site diel baseline, although many such signals have strong day–night cycles. River networks have a specific spatial structure along the flow [36].

Store-and-forward and energy: Node-side buffering is an established practice [4], [5], [8], [37], [38], as is gateway-side storage during backhaul outages [9]; reports also note that draining a backlog can starve live traffic [6] and that storage fails [38]. LPWAN energy rests on measured LoRaWAN [39] and NB-IoT [40], [41] profiles and models [42], [43]. Dual prediction transmits only when the server's prediction would leave a tolerance [44], [45], and it was recently combined with adaptive sampling under an energy budget [46] and given thresholds that adapt to the behaviour of the whole network [47]. It is used to suppress reports over reliable or independently lossy links. The closest treatment of suppression under loss is BaySail [48]: a missing report may be a suppression or a failure, and the server resolves the ambiguity by Bayesian inference with redundancy added to the reports. BaySail infers what the server cannot see; the design in this study instead lets the node, which knows which readings were delivered, decide when a batch must leave early.

What is missing is a quantitative comparison of these remedies in terms of energy, completeness, and timeliness under measured gap processes (Table I). This study also reuses dual prediction for a different purpose: to make an hourly batch leave early when the server's estimate would leave tolerance.

III. Data and Models

The following terms are used throughout:

- Reading: one sampling cycle of a node, i.e., the four parameters taken at one instant, which travel together. Time is divided into slots at the reporting cadence, one reading per slot.
- Delivery gap: a run of slots for which no reading reaches the server, regardless of the cause.
- System outage: a delivery gap verified to affect every node of a network (its gateway, backhaul, or server).

Table I
Closest related work. Gap durations: measured from data, reported without measurement, assumed in a model, or simulated. All four: ARQ, buffering, batching, and reconstruction. –: not addressed.

Work	Gap durations	Remedies compared	Energy	Held-out test
Kong [13]	reported	reconstruction	–	–
Deployments [4], [6], [8], [38]	reported	buffering	–	–
Huang [11], [12]	assumed	retransmission	–	–
GRIN, SPIN [14], [15]	simulated	reconstruction	–	–
Arnaiz [46]	–	dual prediction	modelled	–
BaySail [48]	assumed	dual prediction under loss	–	–
FREE [26]	–	scheduled batching	modelled	–
This study	measured	all four	ratios	yes

- Radio loss: a frame lost while the rest of the path works.

Fig. 1 shows the path from node to server, the logic of the proposed node, and where readings can be lost.

A. Delivery records and the cause of gaps

The study uses four records of what a network delivered (Table II):

- InWaterSense (Kosovo): node 1 on the Sitnica river [3], [49], reporting approximately every 611 s from May 2015 to January 2016. The node sends each parameter as a separate message, seconds apart; the messages of one sampling instant are grouped into a reading, and a slot counts as delivered if a reading arrived for it.
- Weija (Ghana): nine parameter streams of one node, October 2018 to March 2019 [2]. Records marked as imputed do not count, and gap statistics use the conductivity stream (32.5-min cadence). Even among the unmarked records, 10.4% of the conductivity values carry more than the three decimals the logger otherwise records (up to 14), a sign of unflagged interpolation. Such values hide some short gaps; therefore, counts of short gaps are lower bounds, whereas long gaps are not affected.
- UVA (USA): ten LoRaWAN sensors reporting every 5 min through three gateways, February 2024 to June 2025 [19], [50]. Missing frames are found from frame counters and classed as system losses if no sensor reached the database in that 10-min bin, as gateway losses if all of the sensor's gateways were silent, and as radio losses otherwise. With bins of 5 to 30 min, the number of system outages ranges from 14 to 8 (short outages merge in coarser bins), the share of system downtime in outages of a day or more from 82% to 84%, and the system share of missing frames from 28.0% to 28.6%; the classification is therefore not an artefact of the bin.
- LoED (UK): uplinks heard by nine gateways [18], [51]. The devices belong to other operators, and a frame counts as lost when no LoED gateway heard it; therefore, only well-heard devices (at least 200 frames, radio loss below 20%; 179 of 1,780) are used, as a proxy for a node's own link. Losses did not coincide across devices beyond what their density implies: a lost frame coincided with a loss of another device at a shared gateway within the same minute in 19.2% of cases, against 18.2% (16.7%–19.5%) under a random time-shift baseline.

Table II
Delivery records. Loss: share of expected readings (InWaterSense, Weija) or frames (UVA, LoED) that did not arrive. Gaps: runs of missing slots or frames. Lost in $\geq$1 d: $S(1\ \mathrm{d})$ in (1). Weija: conductivity stream (71.0%–100.0% over all nine streams). LoED: well-heard devices (radio loss).

Record	Type	Slot	Loss	Gaps	Lost in $\geq$1 d
InWaterSense	water quality	611 s	46.0%	1,619	50.7%
Weija	water quality	32.5 min	15.4%	42	82.9%
UVA	LoRaWAN, 3 GW	5 min	4.5%	36,519	23.6%
LoED	LoRaWAN, 9 GW	varies	9.6%	56,276	–

Both LoRaWAN analyses count a frame once per device and frame counter, regardless of how many gateways heard it. A counter decrease or a jump beyond a threshold (5,000 frames on UVA, 1,000 on LoED) is treated as a reset or rejoin, not as loss. On UVA, the counter decreased only rarely in the whole record ($n = 1$), no counter was reused, and a threshold of 1,000 instead of 5,000 changes the count of missing frames by 2%. On LoED, 81 of the 179 well-heard devices show a reset, and their pooled loss is 10.0% with a threshold of 100 and 10.9% with 1,000.

Neither water-quality record has a node status log; therefore, a gap in which the network was down cannot be distinguished directly from one in which the node produced nothing. Two partial checks are possible. On Weija, the node's other streams share its logger and uplink: of the 916 missing conductivity slots, 84% were silences of the whole node, and in gaps of 6 h or more, only 6% of the slots had another stream delivering. The long gaps are node-wide silences (node, logger, or uplink). InWaterSense's two nodes reached the server through one gateway and one GPRS link [3]. While node 2 was operating (May–August 2015, a window that holds 57% of node 1's missing slots), it was silent in 79% of node 1's missing slots. The share π of node-1 missing slots with a shared cause follows from $P(\text{node 2 silent} \mid \text{node-1 slot missing}) = \pi + (1-\pi)\, b$, where b is node 2's own rate of silence, measured within a day around each gap at slots where node 1 delivered. Gaps of 6 h or more were approximately 74% shared-cause (gateway, backhaul, server, or shared power) and 26% specific to node 1; gaps shorter than 6 h were 63% node-specific, as radio losses would be. Most long gaps are therefore failures of the path to the server, but some may be nodes that produced nothing; Section V treats buffering's recovery as an upper bound.

B. Gap-length models

A record is reduced to alternating up-runs and gaps of $k_1, \ldots, k_n$ slots; let $h(k)$ be the duration of a gap of k slots. Because loss handling depends on where the missing data are, the main statistic is the share of missing slots in gaps of at least duration L,

$$S(L) = \frac{\sum_{i:\, h(k_i) \geq L} k_i}{\sum_{i=1}^{n} k_i}, \tag{1}$$

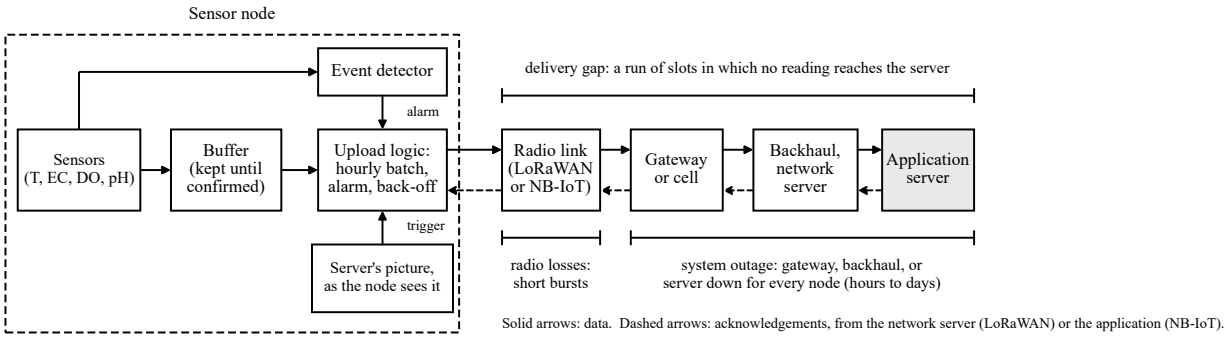


Fig. 1. System model. The sensor node buffers every reading and uploads it through a radio link, a gateway or cell, and a backhaul and network server to the application server, which holds the server's picture; acknowledgements return along the same path. Short radio losses occur on the link; a system outage removes the gateway, backhaul, or server for every node at once.

with bootstrap 95% confidence intervals [52]. The gap lengths are fitted by maximum likelihood with four models. The first is a geometric model,

$$P_{\mathrm{geo}}(k) = (1-q)\,q^{k-1}, \tag{2}$$

the gap-length distribution of independent losses and of the simple Gilbert channel, which loses every frame in its bad state and none in its good state; in the general Gilbert–Elliott channel, where both states can lose frames, loss runs are not geometric. The geometric form follows whenever the loss process has a single loss state with a constant chance of ending in each slot, and it is this assumption that is tested. The second is a two-geometric mixture,

$$P_{\mathrm{mix}}(k) = w\,(1-q_1)\,q_1^{k-1} + (1-w)\,(1-q_2)\,q_2^{k-1}, \tag{3}$$

a Markov chain with one up state and two loss states (short loss, long gap). The third and fourth are a discrete lognormal (4) and a discrete power law (5),

$$P_{\mathrm{ln}}(k) = \frac{\Phi\!\left(\frac{\ln(k+\frac{1}{2})-\mu}{\sigma}\right) - \Phi\!\left(\frac{\ln(k-\frac{1}{2})-\mu}{\sigma}\right)}{1-\Phi\!\left(\frac{\ln\frac{1}{2}-\mu}{\sigma}\right)}, \tag{4}$$

$$P_{\mathrm{pl}}(k) = \frac{k^{-\alpha}}{\zeta(\alpha)}, \tag{5}$$

with Φ the standard normal distribution function and ζ the Riemann zeta function. Models are ranked by Akaike's information criterion [53]. Because AIC only ranks models, each is also tested by a discrete Kolmogorov–Smirnov statistic $D = \max_k |\hat{F}(k) - F(k)|$ with a parametric-bootstrap p-value from 200 samples of the fitted model, each refitted. A gap is bounded by delivered slots on both sides; therefore, every gap length is fully observed, and only the first and last up-runs are censored (they are not modelled).

C. Energy model and loss-handling strategies

The node is a Waspmote-class host [54] with a water-quality sensor board [55] and NB-IoT (the N211 module with release assistance [40]), LoRaWAN SF7, or LoRaWAN SF12 (EU868). Model inputs are listed in Table III, each marked as measured in the cited work, inferred from the records, or assumed, and, where assumed, how it is tested. The main elements are as follows:

- NB-IoT: Energies come from UDP datagrams of 12–512 B sent to an echo server [40]; an unconfirmed report costs 0.31 J (connection released after the uplink) and a confirmed one 0.33 J (released after the server's echo, which is the acknowledgement, end to end). A bulk frame of up to 512 B costs up to 60% more than a 20-B report. No published measurement of the energy of a cell search without a network could be found; it is modelled as the receive power of 0.222 W [41] times a search time T_{search}, swept over 2.2, 10, and 60 s (default 10 s, 2.22 J per failed attempt).
- LoRaWAN: Energies come from state-by-state current profiles [39], with airtime computed from the payload. At SF7, the acknowledgement arrives in the first receive window; therefore, a confirmed success (0.164 J for a 20-B report) is cheaper than an unconfirmed report (0.244 J), which keeps both windows open; at SF12, it arrives in the second window, and a confirmed success is more expensive (0.858 vs 0.729 J). The acknowledgement comes from the network server. LoRaWAN 1.0.2 recommends eight transmissions of a confirmed frame with the data rate lowered every two [56]; LoRaWAN 1.0.4 sends a frame NbTrans times (1–15) at a fixed rate [57]; both are evaluated. A failed confirmed attempt adds a time-out of 1–3 s, and the 1% duty cycle forbids a new frame before 100 times the previous frame's airtime has passed [58].
- Frames: A reading in a bulk frame takes 12 B (four parameters, a time offset, and a sequence number) and a frame 4 B of overhead; therefore, a frame holds 42 readings on NB-IoT, 18 at SF7, and 3 at SF12.
- Host and sensing: The host is awake for every attempt, drawing 63 mW (17 mA [54]); the sensor board is switched off during radio attempts, which its library allows. While sampling, the node draws the host's active current plus the board's constant 1.6 mA and the circuits of the operating sensors (T 3.5 mA, EC 2.5 mA, DO 0.16 mA, pH 0.17 mA [55]), 92 mW in total, for 5 s per sample (an assumed probe warm-up).

Table III
Model inputs: measured (M) in the cited work, inferred (I) from the records, or assumed (A); where assumed, how it is tested.

Input	Value		Tested by
Link state	from the delivery records	I	gap cause, timing models
Retry timing in a gap	full slot, restoration, burst	A	all three
Reading generated in a gap	yes	A	node-off share 0–50%
Radio loss	0 or 0.096 per attempt (LoED)	I	both
Cadence	611 s, 32.5 min, 5–60 min (USGS)	I	–
Payload	20 B report; 12 B per reading + 4 B	A	–
ACK layer	LoRaWAN: network server; NB-IoT: application	M	application ACK variant
Transmissions	8 (LoRaWAN 1.0.2), NbTrans 3 or 8 (1.0.4); NB-IoT 3	M	both versions
Radio energies	state currents and per-report measurements [39], [40]	M	$\times$0.5–2
NB-IoT search	0.222 W $\times$ 2.2–60 s	A	sweep
Host	63 mW awake, 0.12 mW asleep [54]	M	$\times$0.5–2
Sensing	92 mW [55] for 5 s per sample	M, A	ratios unaffected
Storage	unlimited, free	A	RAM 2–64 KB, flash
Backlog drain	immediate	A	duty-cycle limit
Back-off cap K	6 h	A	1–24 h
Battery	6,600 mAh, ideal	A	ratios only

Over a record of D days, the transmission energy per day and the lifetime on a battery of energy B (6,600 mAh at 3.7 V, ideal) are

$$E_{\mathrm{tx}} = \frac{1}{D}\sum_{a} e_a, \qquad Y = \frac{B}{365\,(E_{\mathrm{floor}} + E_{\mathrm{tx}})}, \tag{6}$$

where e_a is the energy of attempt a (host time included) and E_{floor} is the daily sleep and sensing energy: the radio sleeps at 9.35 μW (NB-IoT power-saving mode) or 148 μW (LoRa radio), and the host sleeps at 0.12 mW (deep sleep, 33 μA). Absolute energy depends on the platform and stack: an independent measurement on another board [43] reports 3$\times$ this model's per-message energy on LoRaWAN and 41$\times$ on NB-IoT (TCP without release assistance). The results are therefore ratios between strategies on the same radio; lifetimes are illustrative.

Five strategies are compared:

- Unconfirmed: one attempt per reading.
- Unconfirmed + back-off (NB-IoT): the fair baseline, because a node that searches for a cell in every slot of a gap wastes energy.
- ARQ: up to eight transmissions on LoRaWAN and three on NB-IoT.
- ARQ + buffer: a failed reading is buffered, and the node sends one small confirmed probe at a time, waiting

$$w_n = \min\left(2^n,\, K\right) \text{ slots}, \qquad K = 6 \text{ h}, \tag{7}$$

  after the n-th failed probe and uploading the buffer in confirmed bulk frames once a probe succeeds.
- Batching: the buffer is uploaded every 1 or 6 h with the same back-off.

Replay: A record shows only that the reading at a slot's first instant was lost; what happens to later attempts in that slot is not in the record. Retries happen at their real times, after the receive windows, the time-out, and the duty-cycle wait. Three models bracket the unknown:

- Full slot: the link stays down for the whole slot (pessimistic for ARQ).
- Restoration: the link returns at a uniformly random time before the next delivered slot; therefore, only a gap's last slot can gain from a retry.
- Burst: in addition, gaps of up to an hour are radio-loss bursts through which every retry fails with the probability of a loss following a loss on LoED (0.25), rather than surely (optimistic for ARQ).

Independent radio losses of 0.096 per attempt (LoED) can be added.

D. The server's real-time picture and the tolerances

Let $x(t)$ be the true reading of one parameter at slot t, $A(\tau)$ the set of slots whose readings have arrived by time τ, and $s^*(\tau) = \max A(\tau)$. The server shows

$$\hat{x}(\tau) = \begin{cases} x(\tau), & \tau \in A(\tau), \\ x(s^*) + \Delta(\tau), & \text{otherwise}, \end{cases} \tag{8}$$

where $\Delta(\tau) = x(\tau - 24\,\mathrm{h}) - x(s^* - 24\,\mathrm{h})$ if both readings have arrived and $\Delta(\tau) = 0$ otherwise. That is, the server shows the latest reading plus the site's own change over the same hours a day earlier (the own-diel estimator). The picture is scored as the share of slots with $|\hat{x}(\tau) - x(\tau)| \le \delta$. At the reference level L2, δ is 0.2 °C (T), 2% saturation (DO), 0.1 (pH), and 2% of the median (EC); L1 halves and L3 widens these. L2 lies at or inside the U.S. Geological Survey's rating of an "excellent" continuous record [59] for every parameter: over the sites used, the median L2 tolerance is 0.20 °C against 0.20 °C for T, 2.0 against 4.8 %sat for DO, 4.2 against 6.3 μS/cm for EC, and 0.10 against 0.20 for pH. All three levels are small compared with the median daily range at these sites (T 1.10 °C, DO 10.1 %sat, EC 12.0 μS/cm, pH 0.25).

River data: River signals are approved USGS instantaneous values of T, EC, DO, and pH [60], with DO converted to percent saturation [61] at the site's mean barometric pressure, estimated from its altitude with the standard atmosphere. Section VI uses nine river chains (27 sites) in 2023. Section VII uses every candidate site with all four parameters in complete cycles for at least half of the year: 30 of 39 in 2023 (development) and 33 in 2024 (held out).

IV. How the Records Lose Data

In the water-quality deployments, most missing data were in long gaps. InWaterSense node 1 had 1,619 delivery gaps (timelines in Fig. 2). Of these, 47.3% lasted a single slot but held only 4.7% of the missing readings, while 14 gaps of a day or more held 50.7% (95% CI 29.4%–64.6%), and gaps of 6 h or more held 68.5% (Fig. 3); the longest lasted 13.3 days. Weija, on another continent, showed the same structure: 82.9% of the missing conductivity readings fell in gaps of a day or more (71.0%–100.0% across all streams), with the longest lasting 36.7 days. With few day-long gaps ($n = 3$), the Weija interval is wide (0.0%–93.5%); therefore, Weija serves as qualitative support.

The geometric model failed, and no tested model fitted. The geometric model (2) ranked last by AIC on every record

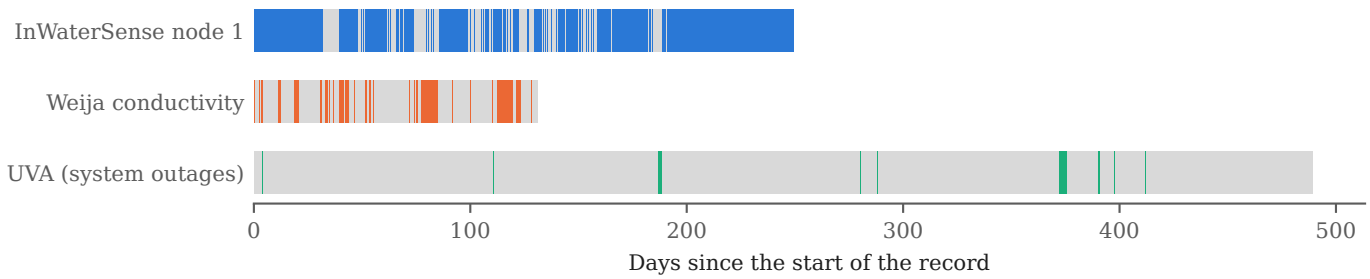


Fig. 2. Delivery timelines of the three records with long gaps: coloured spans are slots in which nothing was delivered (UVA: system outages only; its short radio losses are not shown). Each record is drawn on its own time axis, from its first slot.

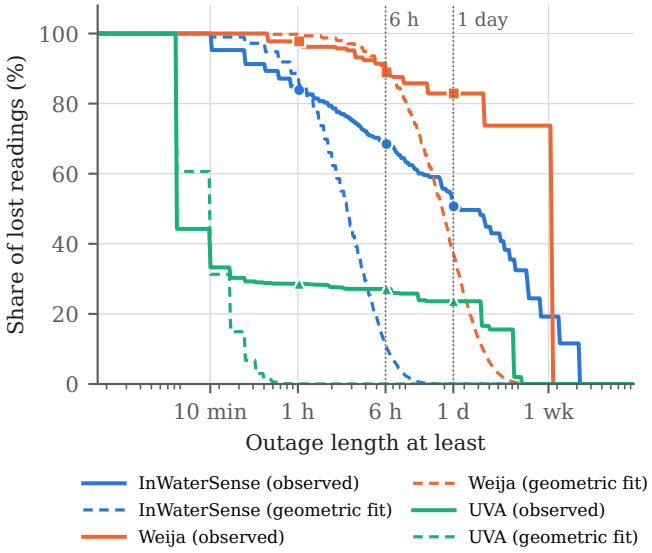


Fig. 3. Share of missing readings in gaps at least as long as the horizontal axis, $S(L)$ in (1) (solid), and as predicted by the geometric model (2) fitted to each record (dashed). InWaterSense node 1 (611-s slots), Weija conductivity (32.5-min slots), and UVA (5-min slots; radio and system losses together).

(Table IV), by 3,677 points on InWaterSense, 120 on Weija, and 43,915 on UVA. It also failed in the quantity that matters: fitted to InWaterSense, it put 11.4% of the missing readings in gaps of 6 h or more, against 68.5% observed, and less than 0.001% in gaps of a day or more. It was also rejected absolutely on every record ($p \leq 0.005$). On the large records (1,619 gaps on InWaterSense, 36,519 runs on UVA), every model was rejected, but the geometric model's largest distance between the fitted and observed distribution functions was 0.48 on InWaterSense and 0.26 on UVA, against 0.027 and 0.008 for the best alternative; on Weija's few gaps, the lognormal and power-law models were not rejected ($p = 0.37$ and 0.64). No single alternative won everywhere; the mixture (3), with many short losses and a few long gaps, is the common thread. On InWaterSense, long gaps also clustered: a long gap tended to be followed by another long gap (Spearman $\rho = +0.27$ between the lengths of consecutive gaps; $p < 0.001$) and by a short up-run ($\rho = -0.20$), which a renewal model that draws each gap independently cannot reproduce. On Weija,

Table IV
AIC of gap-length models (lower is better; best per record in bold). UVA: all missing-frame runs, 5-min slots.

Model	InWaterSense	Weija	UVA
Geometric (memoryless)	10,537	341	76,881
Two-geometric mixture	7,107	236	**32,966**
Discrete lognormal	**6,859**	222	33,327
Discrete power law	6,863	**221**	33,180

neither correlation was distinguishable from zero ($\rho = +0.02$, $p = 0.88$). Therefore, all simulations replay the real records rather than a fitted model.

The LoRaWAN networks showed short radio losses and rare system outages. On LoED, well-heard devices lost 9.6% of their frames to the radio, in short bursts: of 56,276 bursts, 84% were a single frame and 99% were at most five frames, and a loss after a loss had a probability of 0.25. UVA adds 16 months of continuous operation. The system was available 98.72% of the time; of the 58,211 missing frames, 71.6% were radio losses and 28.4% were system losses, and 83% of the system downtime fell in outages of a day or more. One gateway was down for 29.9 days without causing a single gateway loss, because every sensor was also heard by a second gateway, and rain had no detectable effect on link loss (9.18% dry, 9.21% rainy; Wilcoxon [62] $p = 0.64$).

In response to RQ1, in the four records, losses fell into short, weakly bursty radio losses and rarer long delivery gaps or system outages. In the water-quality deployments, gaps of 6 h or more held most of the missing data (68.5% on InWaterSense, 88.9% on Weija). A geometric gap-length model missed this second regime almost entirely.

V. Retransmit or Backfill?

The InWaterSense and Weija records were replayed as the link state for NB-IoT (search times 2.2, 10, 60 s), LoRaWAN SF7, and LoRaWAN SF12, with and without independent radio losses: 20 configurations in all. Table V shows InWaterSense under the full-slot model; ranges are over all configurations.

Table V
Loss-handling strategies on the InWaterSense record (full-slot model, no added radio loss): transmission energy per day, E_{tx} in (6), per radio (NB-IoT: $T_{\mathrm{search}} = 10$ s), and the share of readings delivered and delivered within 1 h (LoRaWAN SF7; NB-IoT for the NB-IoT-only strategy). ARQ: LoRaWAN 1.0.2 behaviour. Back-off (7) capped at 6 h. –: not applicable.

	Energy (J/day)			Readings	
Strategy	NB-IoT	SF7	SF12	Delivered	Within 1 h
Unconfirmed, every reading	224.6	59.3	143.4	53.9%	53.9%
Unconfirmed + back-off (NB-IoT only)	57.7	–	–	47.1%	47.1%
ARQ (8 attempts LoRaWAN; 3 NB-IoT)	226.2	430.5	773.9	53.9%	53.9%
ARQ, 3 attempts (LoRaWAN)	–	168.6	344.7	53.9%	53.9%
Confirmed + buffer + back-off	62.0	50.5	154.8	100.0%	52.8%
Hourly batch + buffer + back-off	23.0	18.2	87.4	100.0%	52.0%
6-hourly batch + buffer + back-off	13.0	10.9	77.2	100.0%	11.2%

A. ARQ pays for losses it can hardly recover

Under the full-slot model, ARQ delivered exactly what unconfirmed reporting delivered, at 2.59×–7.36× its energy on LoRaWAN with eight transmissions (1.51×–2.94× with three); on InWaterSense at SF7, this is 430.5 J/day instead of 59.3 J/day (modelled lifetime 0.46 instead of 1.63 years). This is the pessimistic end of a bracket. If the link may return during a gap's last slot, retries recovered 0.1–4.6 percentage points of InWaterSense's readings on LoRaWAN; if, in addition, gaps of up to an hour are radio-loss bursts, they recovered 6.5–7.7 points. Against the 46 points that unconfirmed reporting lost, ARQ thus recovered under a fifth in every model, at 2.3×–7.3× the energy of unconfirmed reporting; on Weija, it recovered at most 0.9 points. With independent radio losses added, ARQ did recover them, raising the delivered share by $+5.0$ to $+7.9$ points. NB-IoT ARQ recovered nothing in any timing model because the modem reports that there is no network and the node stops retrying. LoRaWAN 1.0.4 (eight transmissions at a fixed rate) changed the price only slightly: 6.8× instead of 7.3× at SF7. Retransmission suits the radio regime, not the long gaps that hold most of the missing data.

B. Buffering buys completeness at a modest energy cost

On NB-IoT, searching for a cell in every slot of a gap cost 1.20×–6.54× the energy of backing off (the ratio grows with the share of time in gaps and with T_{search}); thus, the fair NB-IoT baseline backs off. Against the fair baseline, confirmed reporting with a buffer and back-off delivered every generated reading at an energy change of -5.5% to $+15.4\%$ on NB-IoT and -14.9% to $+25.9\%$ on LoRaWAN. It was no more expensive at SF7 (-14.9% to 0.0%), where a confirmed success costs less than an unconfirmed report and the node stops transmitting into gaps, and more expensive at SF12 ($+8.0\%$ to $+25.9\%$); on InWaterSense at SF7, it used 50.5 J/day (1.73 years). This completeness rests on three assumptions, each of which was tested:

1) The reading must exist: With up to half of the long-gap slots reassigned to a node that was off, buffering still delivered every generated reading, but only 83% of the expected readings on InWaterSense.
2) The buffer must hold the backlog: The longest backlog was 22 KB; 2 KB of RAM lost 24% of the readings, 8 KB lost 10%, and 64 KB none, and writing every reading to serial flash [63] cost at most 0.03 J/day in the datasheet's worst case, against at least 8 J/day of radio energy.
3) The drain must leave room for live traffic: Limited to what the duty cycle leaves after one live frame per slot, the drain still delivered everything. At SF7, the longest backlog cleared in 64 h; at SF12, the median backlog cleared in 1.2 h, but a backlog persisted for 63 days in InWaterSense's worst period, and readings generated while it drained arrived up to 25 days late (95th percentile); therefore, live readings should go ahead of the backlog [6].

C. Batching is cheaper and costs little timeliness

Hourly batching with buffer and back-off delivered every reading, while the fair per-reading baseline used 1.33×–3.31× as much energy. Its share of readings within 1 h differed from that of unconfirmed reporting by only -3.3 to $+7.3$ points: under long gaps, the gaps, not the batch interval, decide how many readings arrive within the hour. The back-off cap trades energy against how long the server waits once the network is back. Probing every slot cost 2.2× the energy of $K = 6$ h at SF7, while raising K to 24 h saved only 12% more and tripled the 90th-percentile wait (20 to 61 h); $K = 6$ h sits at this knee. A better schedule could gain little: a probing schedule fitted to each record's own gap lengths (a lognormal fitted on half of the gaps and scored on the other half, the optimal-checking problem of Barlow et al. [64]) saved at most 2.9% of the design's daily energy on LoRaWAN at the same detection delay, and up to 9% on NB-IoT (InWaterSense record), a figure that rests on the unmeasured cell-search time. The price of batching is the timeliness of alarms and of the real-time picture of fast parameters, which Section VII addresses.

D. Many nodes and model uncertainty

A system outage ends for every node behind the gateway simultaneously. A discrete-event model with one EU868 gateway and up to 500 nodes that buffered through a common outage followed these rules:

- Uplinks on the same channel and spreading factor collide unless one is 6 dB stronger; the collision rule reproduces the pure-ALOHA success probability e^{-2G} in a test.

Table VI
N nodes recovering from a common one-day outage behind one gateway (probe jitter up to 1 h; mean of 3 runs; 72 h horizon). Cleared: nodes whose backlog was delivered; median and last clearance (h after the outage); PDR: uplinks received and acknowledged; collided; received but unacknowledged; node energy over the horizon relative to a node recovering alone.

	N	Cleared	Med. (h)	Last (h)	PDR	Coll.	No ACK	Energy
SF7	10	100%	5	6	99%	0%	1%	1.1×
SF7	50	100%	5	10	97%	1%	1%	1.5×
SF7	100	100%	5	13	93%	1%	3%	1.9×
SF7	250	100%	6	21	85%	2%	9%	3.3×
SF7	500	100%	8	42	77%	2%	15%	7.2×
SF12	10	100%	8	14	92%	2%	2%	1.3×
SF12	50	100%	15	43	67%	7%	16%	3.7×
SF12	100	93%	26	>72	47%	13%	25%	7.1×
SF12	250	36%	>72	>72	38%	18%	28%	4.7×
SF12	500	13%	>72	>72	32%	21%	29%	3.1×

- The gateway demodulates at most eight uplinks simultaneously and cannot receive while it transmits.
- The gateway acknowledges in the first receive window if its 1% sub-band allows, otherwise in the second (869.525 MHz, SF12, 10% sub-band), and otherwise not at all.
- Nodes back off as in (7), drain with up to eight transmissions per frame, and respect their own duty cycle.

Table VI gives the results, which showed two effects. Without randomisation, the deterministic back-off kept the nodes synchronised: from 50 nodes at SF7, almost none recovered within 72 h. With probes delayed randomly by up to an hour, every SF7 node recovered, the last within 42 h, at up to 12.5× the energy of a node alone. At SF12, the gateway's acknowledgements ran out: its first-window sub-band reached its 1% limit, up to 29% of the received frames went unacknowledged, and after a one-day outage, only 93% of 100 nodes cleared their backlog within 72 h. This is mainly a steady-state limit of confirmed traffic in large SF12 cells [23], [24], not an effect of the outage. In a ten-day run of the same model, 100 SF12 nodes whose outage lasted only 15 min held 740–870 buffered readings from the first day to the tenth, with approximately a third of the received frames unacknowledged. After a one-day outage, the backlog of 9,600 readings fell to the same level (863) within a week. The outage added a backlog that the acknowledgement-limited cell cleared slowly; it did not create a lasting failure. With 25 and 50 nodes, the backlog returned to a few readings per node within 72 h of either outage, and with 200 nodes, it grew without bound after either. The per-node results hold for cells with tens of nodes.

With every measured energy component (the LoRaWAN and NB-IoT reports, the acknowledgement time-out, the bulk-size slope, the search power, and the host's awake power) multiplied by an independent log-uniform factor between 1/2 and 2, and the NB-IoT search time drawn between 2.2 and 60 s (100 draws), ARQ cost more than unconfirmed reporting on LoRaWAN in 100% of the draws (a ratio of at least 1.8× in 97.5% of them) and approximately the same on NB-IoT (median 1.01×). Hourly batching was cheaper than the fair baseline in at least 98% of the draws at SF7 and on NB-IoT, but only in 85%–91% at SF12. The buffering energy relative to the fair baseline ranged from 0.49× to 2.12×, and its sign is set by the ratio of confirmed to unconfirmed energy, which can only be settled by measuring the target hardware.

In response to RQ2, ARQ recovers only the short radio losses under any timing model, and on LoRaWAN, it multiplies energy. Buffering with back-off recovers every generated reading, given tens of kilobytes of storage, at an energy -14.9 to $+25.9$% different from that of recovering nothing on LoRaWAN (-5.5 to $+15.4$% on NB-IoT), and hourly batching is the energy-efficient way to do it at SF7 and on NB-IoT. Many nodes recovering simultaneously need randomised probes, and large SF12 cells are limited by the gateway's acknowledgements.

VI. Can Reconstruction Replace Delivery?

If a server could estimate missing readings sufficiently well, it would not need them delivered. This was tested on nine river chains, i.e., sets of USGS sites along the same river (27 sites), with each pair of sites serving as target and neighbour in both orders. On a 1-h grid, gaps of 1, 3, 6, 12, 24, 48, and 72 h were simulated at the target in the last 80% of 2023 (a median of 300 per pair, parameter, and length). A gap covers the slots after the last observed slot t_0 and before the next observed slot t_1. Let y be the neighbour's series, lagged by ℓ hours, and b a scaling slope; the lag $\ell \in [0, 48]$ h maximises the correlation between 6-h changes of the target and the lagged neighbour in the first 20% of the year, and b is the least-squares slope of those changes (pairs whose best correlation is negative are dropped). Two families of estimators were compared. Real-time estimators use only data up to t_0 and the neighbour during the gap:

$$\hat{x}_{\mathrm{pers}}(t) = x(t_0), \tag{9}$$

$$\hat{x}_{\mathrm{diel}}(t) = x(t_0) + x(t - 24\,\mathrm{h}) - x(t_0 - 24\,\mathrm{h}), \tag{10}$$

$$\hat{x}_{\mathrm{nb}}(t) = x(t_0) + b\,[y(t) - y(t_0)], \tag{11}$$

$$\hat{x}_{\mathrm{dn}}(t) = \hat{x}_{\mathrm{diel}}(t) + b\,[y(t) - y(t_0)] - b\,[y(t - 24\,\mathrm{h}) - y(t_0 - 24\,\mathrm{h})]. \tag{12}$$

Persistence (9) holds the last value; own diel (10) is the server's estimator (8); neighbour change (11) adds the neighbour's scaled change since the gap began; and diel + neighbour (12) adds only the part of that change that the neighbour's own diel pattern does not explain. Bridging estimators also know $x(t_1)$, as after a backfill:

$$\hat{x}_{\mathrm{int}}(t) = x(t_0) + \frac{t - t_0}{t_1 - t_0}\,[x(t_1) - x(t_0)], \tag{13}$$

$$\hat{x}_{\mathrm{in}}(t) = \hat{x}_{\mathrm{int}}(t) + b\,[y(t) - \bar{y}(t)], \tag{14}$$

where $\bar{y}$ is the straight line from $y(t_0)$ to $y(t_1)$; linear interpolation (13) is the first bridging estimator, and (14) adds the neighbour's departure from its own line. The error on a gap is its maximum absolute error; confidence intervals resample whole chains, and pairs within them. Relative gains exclude the pH pairs ($n = 3$) whose target (site 14206694) has a median error of exactly zero. The neighbours lie 2.8 to 300 km from their targets (median 42 km), with the order along the river taken from the drainage area in the USGS site record.

Table VII
Reconstruction on nine river chains: median over ordered site pairs of the median maximum absolute error over a gap (T in °C, DO in %sat, pH in pH units, EC in μS/cm), and the share of pairs in which the neighbour improves on the target's own data. Real time: best neighbour method vs the better of persistence and own diel. Bridging: interpolation + neighbour vs interpolation.

		Real time				Bridging		
Param.	Gap (h)	Persis-tence	Own diel	Best neighbour	Neighbour wins	Interpolation	Interp. + neighbour	Neighbour wins
T	1	0.07	0.05	0.05	23%	0.03	0.03	19%
T	24	0.97	0.70	0.65	72%	0.80	0.56	93%
DO	1	0.67	0.50	0.60	19%	0.26	0.35	13%
DO	24	9.22	4.41	5.27	30%	7.83	6.35	70%
pH	1	0.000	0.025	0.018	7%	0.013	0.013	9%
pH	24	0.175	0.125	0.149	9%	0.172	0.172	38%
EC	1	0.50	0.50	0.66	16%	0.25	0.27	16%
EC	24	7.50	8.25	7.50	23%	4.44	4.42	25%

In real time, a site's own diel pattern was the estimator to beat (Table VII). For day-long gaps, it beat persistence in 77% of pairs for DO, and a neighbour improved on the better own-data estimator in only 30% of pairs for DO, 9% for pH, and 23% for EC; temperature, with its shared weather driver, was the exception (72%). When bridging, the neighbour helped T (93% of pairs) and DO (70%), but not pH or EC.

A naive baseline overestimates the value of neighbours. Against persistence alone, a neighbour won in 77% of DO pairs, cutting the median error from 9.22 to 5.27 %sat; however, the own-diel estimator already reached 4.41 %sat. Much of what a naive test credits to spatial correlation is the shared day–night cycle. The fitted relation also did not follow the flow: a downstream neighbour had a positive lag approximately as often as an upstream one (64% vs 68% for DO); therefore, the lag tracks shared timing rather than travel time. Evaluations of spatio-temporal reconstruction on rivers therefore need a per-site diel baseline and spatial structure from the river network [33], [36].

No estimator kept day-long gaps reliably within tolerance. For 1-h gaps, the better own-data estimator kept 96% of gaps within the L2 tolerance; for 24-h gaps, the best of all six kept 31% (95% CI 18%–49%), from 9% for T to 49% for pH.

A. Contemporary deep imputers

To test whether this is a limit of reconstruction or of simple estimators, SAITS, a self-attention imputer [34], and StemGNN, a graph neural network [32] (PyPOTS implementations), see all four parameters of every site of a chain in 192-h windows, normalised per window and series from the observed values. They were trained per chain on the first half of 2023, with the library's random point masks and, in half of the windows, a whole site masked for 1–72 h; the checkpoint with the best error on whole-site blocks in the last 15% of that half was kept. StemGNN used one spectral block, one stack, and width 16 (7.5 million parameters). A first configuration, normalised with half-year means and spreads, under-fitted by an order of magnitude even on 1-h gaps and was discarded before any comparison. The models were tested on whole-site gaps of 1, 6, 24, and 72 h in the second half, the same 48,455 cases on which the simple estimators were scored, with the neighbour chosen and fitted on the first half. GRIN and SPIN need a graph library that the available machine could not hold, and ImputeFormer did not finish training in the available time. The deep imputers kept fewer gaps within tolerance: 65% (SAITS) and 63% (StemGNN) of 1-h gaps when bridging, against 97% for interpolation, and 25% and 24% of day-long gaps, against 31%. In real time, they kept 18% and 16% of day-long gaps, against 29% for own diel. The one exception was bridging a day-long gap, for which SAITS had the lower median error: 4.5 %sat for DO and 0.126 for pH, against 6.9 and 0.148 for interpolation. No estimator, simple or deep, kept more than 55% of the day-long gaps of any parameter within tolerance.

In response to RQ3, the evaluated estimators, simple or deep, cannot replace delivery for the day-long gaps that hold most of the missing data. Reconstruction plays the role of the interim estimate between deliveries, which is the role it plays in the design of Section VII.

VII. A Reporting Design and its Evaluation

Sections IV–VI point to a design: buffer and back off through gaps, batch routine data, and show the site's own diel estimate between deliveries. Batching leaves two weaknesses: alarms and the real-time picture of fast parameters. The design closes them with two triggers that send a batch early.

A. The design

Event-driven batching buffers every reading and uploads the buffer once per hour in confirmed bulk frames. When the node's event detector raises an alarm, the buffer is uploaded immediately. A failed upload puts the node into an outage mode, in which an alarm triggers an immediate probe outside the back-off schedule (7); Fig. 4 shows the behaviour in a short scenario. Before the outage, the alarm makes the event-driven node upload immediately, where plain batching would wait for the end of the hour; the alarm during the outage cannot get through; and after the outage, when the backlog arrives depends on where each strategy is in its back-off.

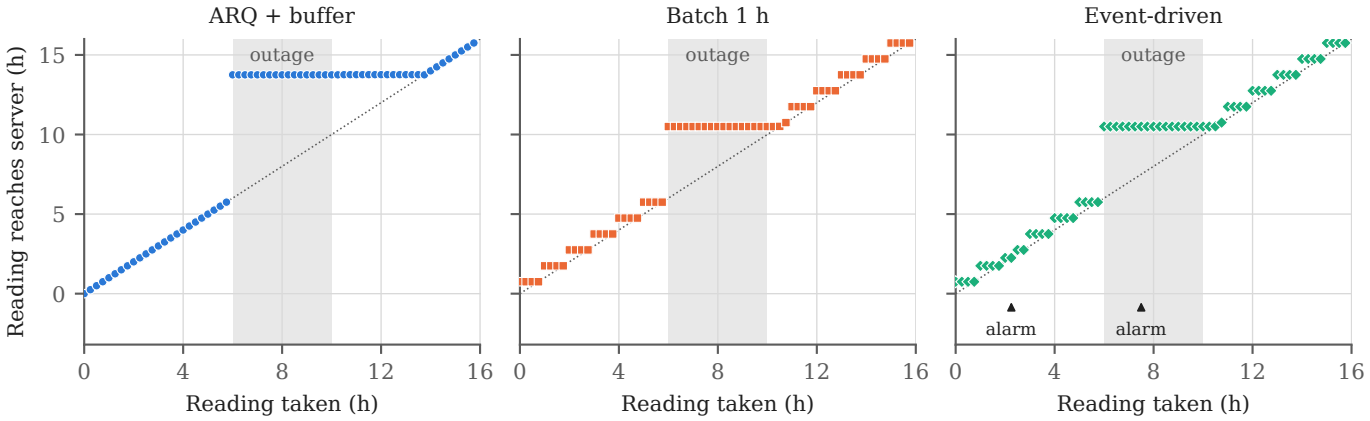


Fig. 4. When each reading reaches the server, in an illustrative scenario run through the simulator: LoRaWAN SF7, 15-min sampling, the link down from 6 to 10 h, and node alarms at 2.25 h (link up) and 7.5 h (link down). Points on the dotted diagonal arrive immediately; horizontal runs are readings delivered together. Event-driven batching follows hourly batching, except that the alarm at 2.25 h sends its batch early.

Bounded event-driven batching adds a bounded-error trigger (Algorithm 1). Because every upload is confirmed, the node knows which readings have reached the server and when; therefore, it can compute the value $\hat{x}_j(t)$ that the server shows for each parameter j by evaluating (8) on those readings. It uploads immediately if

$$\max_j \frac{|x_j(t) - \hat{x}_j(t)|}{\delta_j} > 1, \tag{15}$$

that is, if any parameter's true reading differs from what the server shows by more than its tolerance δ_j. This is the dual-prediction mechanism [44], [45], used to bound the error between batches rather than to suppress reports. Three details matter:

1) The trigger is not consulted in outage mode, where the estimate may drift far and probing would waste energy on a dead link; only alarms probe.
2) The node's copy of the server must model when readings arrive. The duty cycle spaces the frames of a long upload; therefore, a reading counts only after its frame's arrival, and the newest valid reading goes in the first frame.
3) The node's knowledge is only as good as its acknowledgements. On NB-IoT, the confirmation is the application's reply; on LoRaWAN, it comes from the network server, and forwarding to the application server, which UVA shows can fail, is not covered. A variant closes this gap by keeping the bound for both possible server states until an application acknowledgement returns in a later downlink (Section VII-C).

Algorithm 1 Bounded event-driven batching: node, one slot

append the slot's reading x_t to the buffer
if in outage mode **then**
  if alarm at t **or** back-off timer expired **then**
    probe; on success leave outage mode and upload the buffer
  end if
  return
end if
$\hat{x}(t) \leftarrow$ server's estimate (8), from readings whose arrival time is $\leq t$
if alarm at t **or** trigger (15) **or** hour boundary **then**
  upload the buffer (newest valid reading first), confirmed bulk frames
  on failure enter outage mode, back-off (7)
end if

B. Evaluation protocol

Two phases: The design was developed on 2023 data and tested on held-out 2024 data. In development, hypotheses H1–H4 and their decision rules were written before their runs; after H4 failed, H5–H10 were written for the bounded-error trigger before it was implemented or run. Five deviations were recorded when made:

- D1: UVA replayed as system outages plus radio losses.
- D2: one site excluded because its eligibility had been computed over its record span rather than the year.
- D3: seeds derived from site identifiers.
- D4: two implementation defects of the packing order, which made H8 fail as written (below).
- D5: a floating-point tie at the tolerance, found while the held-out runner was being built and before any 2024 data were analysed. The pH and T records are in 0.1 steps, and an error of exactly one tolerance was counted in or out by rounding noise, in the scores and in the trigger.

All development results are from a rerun with D5 fixed; in the first development run, H4 had also failed for pH, a failure caused by D5, not by batching. For the held-out phase, the final design, the analysis code, its decision rules, and the site-eligibility rule were frozen, and the code was identified by cryptographic hashes before any 2024 data were downloaded. The plans were kept with the code; they were not registered with a third party.

Post hoc correction: After the held-out run, an audit of the model inputs found two errors. The energy model charged sensing at the host's power with the sensor board on, omitting the temperature and conductivity circuits, and charged the

board during radio attempts; DO saturation was computed at sea-level pressure, which understated DO in percent saturation by a factor of 1.13–1.16 at the high-altitude sites ($n = 4$) and thus loosened their DO tolerance. Both were corrected, and every run of both years was repeated with the frozen analysis code. No verdict changed. Table VIII reports the verdicts of the frozen run, and every number in the text comes from the corrected runs.

Setting: The evaluation used 30 sites in 2023 and 33 in 2024 (29 in both), sampled every 5–60 min.

- Links: no outage, and the InWaterSense, Weija, and UVA records, mapped onto each site's time grid in physical time, wrapped circularly with a random start offset per site (seeded by the site identifier, with a different seed in 2024); UVA as its system outages plus independent radio losses at the rate of a representative sensor (3.08% per attempt). The first 20% of each series trains the detector, and all metrics use the remaining 80%.
- Radios: NB-IoT, LoRaWAN SF7, and LoRaWAN SF12.
- Strategies: unconfirmed, ARQ + buffer, hourly batching, event-driven batching, and bounded event-driven batching (L2, unless stated otherwise).
- Events: steps and ramps of 5 and 8 robust standard deviations, lasting 3 h, injected one at a time into the real series (7,548 in 2024), as described below. All strategies use the same causal CUSUM detector, fixed before any evaluation.

Event injection: For parameter j, let $a_j(t)$ be the reading minus its centred 24-h running median, and $r_j(t)$ that anomaly minus its median for the same calendar month and hour of day. The event scale is

$$\sigma_j = \max\left(1.4826\ \mathrm{MAD}[r_j]\,,\ \rho_j\right), \tag{16}$$

floored at the recording resolution ρ_j, the most common positive difference between sorted distinct values. An event of amplitude $\kappa\sigma_j$, with σ_j from (16), $\kappa \in \{5, 8\}$, and a random sign, lasts 180 min: a step adds $\kappa\sigma_j$ throughout, and a ramp rises linearly from 0 to $\kappa\sigma_j$; values are rounded to ρ_j. Events touch one parameter each, lie in the test period, at least a day after the start of a data segment and wholly inside it, and are at least three days apart (at most 60 per site and setting). Their median amplitudes are mild in physical units (5σ/8σ: T 0.74/1.19 °C, DO 7.6/12.1 %sat, EC 7.4/11.9 μS/cm, pH 0.50/0.80); of the 2024 events, 99.7% stay within T -0.5–40 °C, positive EC, pH 4–11, and DO 0–250 %sat (99.8% in 2023), the latency verdicts also hold on the plausible events alone, and real 3-h changes of at least 5σ occur at 100% of the sites for T, 100% for DO, 93% for EC, and 60% for pH.

Detector: The detector compares each reading with the reading $m = 120$ min earlier, corrected by a learned hour-of-day profile $p(\cdot)$ of the departure from the running 24-h median, and runs a two-sided CUSUM [65] on the clipped, standardised residual:

$$z_t = \mathrm{clip}\left(\frac{x_t - x_{t-m} - p(h_t) + p(h_{t-m})}{s},\ -c,\ c\right), \tag{17}$$

$$S_t^+ = \max\left(0,\ S_{t-1}^+ + z_t - k\right), \quad S_t^- = \max\left(0,\ S_{t-1}^- - z_t - k\right), \tag{18}$$

where h_t is the hour of day and s is a robust residual scale floored at ρ_j. An alarm is raised when either sum in (18), driven by (17), exceeds H, with $k = 3$, $H = 8$, and $c = 6$; the detector then stays in an event state until $|z_t| < 2$ for 60 min. The profile and scale are initialised on the training period and updated by exponential smoothing (memory of approximately 7 days) outside the event state. These parameters were fixed before any evaluation. In 2024, the detector caught a median (over sites) of 67% of 5σ steps, 91% of 8σ steps, 13% of 5σ ramps, and 65% of 8σ ramps (58.2% of all events), with a median of 0.8 false alarms per parameter per 30 days (mean 1.1).

The strategies were compared on three metrics:

- Alarm latency, from event onset to arrival at the server (detected events, link up at onset).
- Transmission energy (6), including alarm and false-alarm uploads.
- The picture, i.e., the share of slots within tolerance on the series without events.

Each is the median over sites per link and radio (12 combinations; 48 with the four parameters), with 95% confidence intervals from resampling sites. Table VIII lists the hypotheses and verdicts.

C. Results

Alarms (H1, H7): With the link up, event-driven batching raised alarms as fast as per-reading ARQ in both years (Table IX); the median latency of approximately an hour is the detector's own. Plain hourly batching was slower in every combination, by 0.43–0.50 h in 2024, and the bounded-error trigger did not delay the alarms. Counting all injected events, the server knew 34% within an hour with event-driven batching on the no-outage link, as with per-reading reporting, against 21% with hourly batching; within a day, it knew 58% with any of them. The ceiling is detection, not reporting.

Energy (H2, H6): Alarms were rare (a median of 0.13 per node per day); therefore, event-driven batching cost 1.00×–1.02× as much as plain batching. The bounded-error trigger added uploads between batches, at 1.05×–1.32× the energy of plain batching, and per-reading ARQ still used 1.34×–2.53× as much energy as the bounded design (every site-resampled 95% interval above 1.28×; 2023: 1.32×–2.47×).

Whole node: Transmission is not the whole budget. With sensing charged at 92 mW for 5 s per sample, transmission was 16%–54% of the bounded design's node energy and 31%–63% of per-reading ARQ's (medians over sites). Per-reading ARQ therefore used 1.15×–1.41× the bounded design's node energy, against 1.34×–2.53× its transmission energy, and the bounded design extended battery life by 15% to 41%. Once readings are batched, the reporting strategy acts on half of the energy budget or less, and the sensing time per sample matters as much as the radio.

Picture (H4, H5): Plain event-driven batching failed H4 in both years, for DO only: an estimate up to an hour old leaves the tolerance between batches, and the share of slots within tolerance differed from that of per-reading reporting by -10.7 to -6.9 points. The failures occurred with no outage

Table VIII
Hypotheses, decision rules, and verdicts. Each rule applies to the median site of every link–radio (12) or link–radio–parameter (48) combination and passes only if it holds in all. Written: P, before the development runs; A, after H4 failed, before the runs concerned. 2023: development rerun with all deviations fixed; 2024: held out, judged by the frozen code. Event: event-driven batching; Bounded: bounded event-driven batching.

		Claim	Rule	2023 (development)	2024 (held out)
H1	P	Link up: Event raises alarms as fast as per-reading reporting	Event − ARQ+buffer alarm latency $\leq$ 1 slot	Pass, 12/12	Pass, 12/12
H2	P	Event costs approximately as much as batching, much less than per-reading ARQ	Event / batch $\leq$ 1.10 and ARQ+buffer / Event $\geq$ 1.3	Pass, 12/12	Pass, 12/12
H3	P	Onset in a gap: buffering delivers late, unconfirmed loses	Descriptive	As expected	As expected
H4	P	Event's picture is as good as per-reading reporting's	Event $\geq$ ARQ+buffer − 5 pp	Fail, 38/48	Fail, 39/48
H5	A	Bounded's picture is as good as per-reading reporting's	Bounded $\geq$ ARQ+buffer − 1 pp	Pass, 48/48	Pass, 48/48
H6	A	Bounded costs no more than per-reading ARQ	Bounded $\leq$ ARQ+buffer energy	Pass, 12/12	Pass, 12/12
H7	A	The trigger does not delay alarms	Bounded − Event alarm latency $\leq$ 1 slot	Pass, 12/12	Pass, 12/12
H8	A	Newest-first packing makes the picture exact with the link up	100% within tolerance, no outage, every site	Fail as written; pass after D4	Pass
H9	A	H5 holds at tolerances L1 and L3	As H5	Pass, 48/48, 48/48	Pass, 48/48, 48/48
H10	A	H6 holds at tolerances L1 and L3	As H6	Pass, 12/12, 12/12	Pass, 12/12, 12/12

Table IX
Held-out year (2024): alarm latency with the link up (median over sites, h) and transmission energy (median over sites, J/day); LoRaWAN SF7. ARQ: ARQ + buffer.

	Alarm latency (h)				Energy (J/day)			
Link	ARQ	Batch	Event	Bounded	ARQ	Batch	Event	Bounded
No outage	1.00	1.25	1.00	1.00	32.6	8.6	8.6	11.0
UVA	1.00	1.25	1.00	1.00	34.5	9.2	9.3	11.7
Weija	1.00	1.25	1.00	1.00	30.8	10.3	10.4	12.4
InWaterSense	1.00	1.50	1.00	1.00	35.6	17.7	17.9	20.5

at all; they are a property of hourly batching. (In the first development run, H4 also failed for pH; those failures were caused by the tolerance tie, not by batching.) The bounded-error trigger removed the failure: its share differed from that of per-reading reporting by -0.01 to $+0.56$ points for every parameter, link, and radio (Fig. 5), and with the link up, it was within tolerance at every slot (H8, after the packing-order fix D4). The bound costs more at tighter tolerances, yet per-reading ARQ used 1.20×–1.57× the bounded design's energy at L1 and 1.40×–3.46× at L3 (H9, H10). Table X gives the picture for every parameter. Under the two water-quality gap records, unconfirmed reporting showed a better picture than ARQ + buffer for every parameter and radio (DO on InWaterSense at SF7: 75.7% vs 72.4%): when the link returns, an unconfirmed node delivers its next reading immediately, while a buffering node waits for its next back-off probe.

Events in a gap (H3): Each strategy must wait for a link. Unconfirmed reporting learned at most 7.0% of the detected events whose onset fell in a gap (Table XI and Fig. 6); the buffering strategies learned at least 98.8% after the gap, and the alarm probe made the event-driven designs faster than ARQ + buffer on the water-quality records.

Table X
Held-out year (2024): share of scored slots within the L2 tolerance (median over sites; LoRaWAN SF7; series without injected events). ARQ: ARQ + buffer.

Link	Param.	Unconf.	ARQ	Batch	Event	Bounded
No outage	DO	100.0%	100.0%	91.8%	91.8%	100.0%
	pH	100.0%	100.0%	99.0%	99.1%	100.0%
	T	100.0%	100.0%	98.7%	98.7%	100.0%
	EC	100.0%	100.0%	96.7%	96.7%	100.0%
UVA	DO	98.8%	99.2%	91.3%	91.5%	99.2%
	pH	99.8%	99.8%	98.5%	98.6%	99.8%
	T	98.9%	98.8%	97.7%	97.7%	98.8%
	EC	99.2%	99.3%	96.5%	96.6%	99.3%
Weija	DO	90.4%	89.6%	83.8%	83.8%	89.4%
	pH	95.3%	95.0%	94.1%	94.1%	95.4%
	T	89.0%	88.2%	86.9%	87.0%	88.2%
	EC	90.5%	89.6%	87.2%	87.2%	89.7%
InWaterSense	DO	75.7%	72.4%	67.7%	67.7%	72.4%
	pH	90.9%	88.3%	90.1%	90.2%	90.6%
	T	74.1%	67.7%	67.2%	67.5%	68.2%
	EC	80.5%	74.6%	74.0%	73.8%	75.1%

Table XI
Held-out year (2024): detected events whose onset fell in a gap (LoRaWAN SF7, pooled over sites): number of events, share the server ever learns, and median latency of those it learns (hours).

		Known ever			Median latency (h)			
Link	n	Unconf.	ARQ	Event	ARQ	Batch	Event	Bounded
InWaterSense	2,113	3.0%	98.8%	98.8%	27.5	26.6	24.3	24.0
Weija	661	4.1%	100.0%	100.0%	53.5	48.5	48.5	48.5
UVA	57	7.0%	100.0%	100.0%	28.5	24.9	24.9	24.9

Packing order (H8): On SF12, a frame holds only a few readings (Section III-C); therefore, an hourly batch at 15-min sampling needs two frames, and the duty cycle delays the second. With first-in-first-out packing, the newest reading

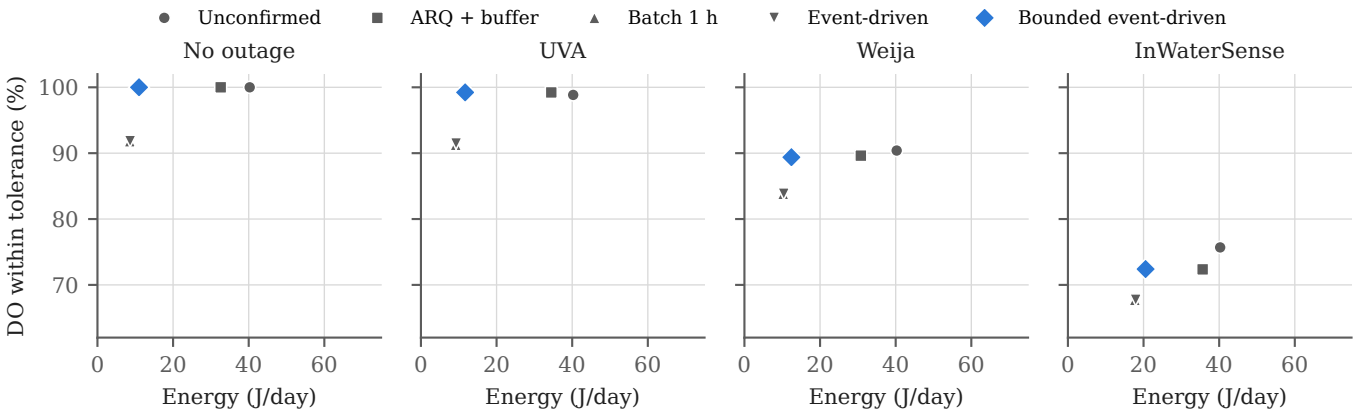


Fig. 5. Held-out year (2024): energy against the real-time picture of DO (share of slots within the L2 tolerance), LoRaWAN SF7, median over sites, one panel per link, and all panels on the same vertical scale. Batch 1 h and event-driven batching nearly coincide, and the bounded design reaches the picture of per-reading reporting at a fraction of its energy.

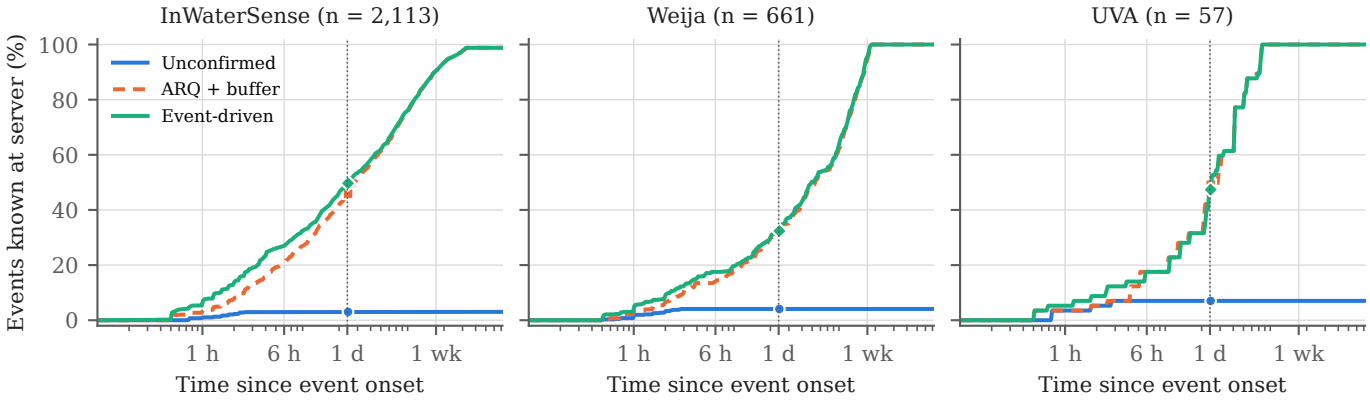


Fig. 6. Held-out year (2024): detected events whose onset fell in a gap, share known at the server by the time on the horizontal axis (LoRaWAN SF7, pooled over sites; markers at 1 day).

rode in the second frame, and in the first development run, the picture on the no-outage link fell to 98.72% at worst. H8 failed as written at one site (01467200, 5-min sampling, SF12; minimum 99.998%) because of two implementation defects: the reading promoted to the first frame could be a placeholder for a sensor gap, and the node's copy of the server counted readings as known as soon as their upload began, although at 5-min sampling an SF12 upload spans several slots. With both fixed (D4), newest-first packing was within tolerance at every scored slot of the no-outage link, on every site, parameter, and radio, in both years, at 0.987–1.007 times the energy of first-in-first-out packing.

Acknowledgement boundary: If a LoRaWAN node may not trust the network server's acknowledgement and keeps the bound for both possible server states, the design cost 3% to 18% more energy on the development sites. Its picture stayed within -0.01 points of per-reading reporting, and per-reading ARQ still used $1.28\times$–$1.99\times$ as much energy.

What the hourly batch buys: Plain dual prediction, which sends only when the server's estimate would leave tolerance [44], [45], is the standard way to send less. A post hoc ablation on the held-out sites removed the hourly batch and kept everything else (the trigger, the alarm uploads, and buffering with back-off). It saved 20%–57% of the transmission energy and changed the picture by at most 1.8 points, but only 26%–55% of the readings then reached the server within 1 h, against 100% with the batch and no outage. The hourly batch is therefore the price of a complete record on the server within the hour; where only alarms and a bounded real-time picture are needed, the trigger alone suffices.

In response to RQ4, the scheme met every criterion with the link up, on data not used for its design. Bounded event-driven batching delivers every generated reading, raises alarms as fast as per-reading reporting, and keeps the picture within tolerance as often, while per-reading ARQ used $1.34\times$–$2.53\times$ as much energy. No scheme fixes the gap itself: data and events during a gap reach the server only when the link returns.

VIII. Discussion

A. Why simple rules suffice

Three results bound what a more elaborate reporting protocol could add under the gaps measured here. First, the timing

of recovery probes is near its limit: a schedule fitted to each record's gap lengths saved at most 2.9% of the design's energy on LoRaWAN (Section V-C); only on NB-IoT, where every failed probe pays an unmeasured cell search, did it reach 9%. Second, the reporting decisions act on a shrinking share of the budget: with hourly batching, transmission was 16%–54% of the node's energy; therefore, even a protocol that halved it would extend battery life far less than the transmission ratios suggest. Third, the parts of the design are separable by what they buy: the bounded-error trigger buys the real-time picture, the alarm upload buys alarm latency, and the hourly batch buys completeness within the hour at 20%–57% of the transmission energy; an operator who does not need the last can drop it. The remaining gains lie outside the reporting protocol: in sensing time and sensor power, in the network that causes the gaps, and in measuring the target hardware.

B. Practical guidance

The results lead to six recommendations:

1) Determine which regime loses the data: Frame counters, a log of gateway and server downtime, and the node's own status are sufficient to distinguish short radio losses from long delivery gaps, which call for different remedies; models that assume geometric gap lengths should be checked against such a log.
2) Do not fight long gaps with retransmissions: Confirm, buffer, and back off; on NB-IoT, back off even without a buffer, because searching for a cell in every slot of a gap cost up to 6.54× the energy of backing off. Give the buffer tens of kilobytes, and at SF12 let live readings go ahead of the backlog; where many nodes share a gateway, randomise recovery probes.
3) Batch, and let the batch leave early when it matters: Hourly batching delivers every reading for a fraction of per-reading energy at SF7 and on NB-IoT; an alarm trigger restores timeliness, and a bounded-error trigger that mirrors the server's estimate restores the real-time picture, with the tolerance as the operator's energy control. The node must model when its data arrive and, on LoRaWAN, whether the application has them. Where the record need not be complete within the hour, the trigger without the batch saves a further 20%–57% of transmission energy.
4) Count the whole node: Once readings are batched, transmission is only 16%–54% of the node's energy; therefore, the probe warm-up time and the sensor board's power deserve as much attention as the radio protocol.
5) Judge reconstruction against the site's own diel pattern, the best simple real-time estimator for DO and pH: A test against the last value alone overstates what neighbours and models add.
6) Engineer the gaps themselves: No reporting scheme shortens a gap. Gateway diversity masked a month-long gateway outage at UVA, and watchdog resets bound silent gateway failures [6], [7].

C. Validity and limitations

The main limitations are as follows:

- Records: The loss evidence comes from four records and supports statements about them, not a law for all deployments; Weija serves as qualitative support, and the cause of a gap could be checked only partly; therefore, buffering's recovery is an upper bound.
- Energy without hardware: No hardware was built for this study. The model rests on published measurements of specific modules, and an independent measurement on another board differs severalfold in absolute terms. The claims are therefore ratios on the same radio, tested against a factor-of-two uncertainty in every component (Section V-D). At SF12, the advantage of batching, and everywhere, the sign of buffering's small energy difference, need hardware measurement, which is the most important next step.
- Model scope: Clock faults and storage failure are not simulated, and the multi-node model of one gateway shows mechanisms (synchronised probes, acknowledgement capacity) rather than numbers for a particular network.
- Events and rivers: Events are synthetic steps and ramps of realistic size, and the detector catches slow ramps poorly (13% of 5σ ramps); the latency comparisons concern detected events. The river data cover two years in one country, and the held-out year shares most sites with the development year (29 of 33); therefore, it tests the design on new data rather than on new rivers.
- Reconstruction: The deep imputers were trained on the first half of one year per river chain, which is a small amount of data for such models, and the neighbours lie 2.8 to 300 km from their targets (median 42 km); denser networks and longer training records may reconstruct gaps better.
- Measurement resolution and pre-specification: pH is recorded in steps of 0.1, equal to its tolerance at the reference level; therefore, part of the trigger's activity and of the picture score for pH responds to one-step flicker rather than to changes in the river. Hypotheses were written before their runs, and the confirmatory code was frozen before the held-out data were downloaded, but neither was registered with a third party; every verdict is reported, including the failures and the defect (D5) found before the held-out run.

D. Conclusion

In the four records studied, data were lost in two regimes: short radio losses and rarer long delivery gaps that, in the water-quality deployments, held most of the missing data. Retransmission spends energy on gaps it cannot bridge, and the evaluated reconstruction estimators, simple or deep, kept most day-long gaps outside the tolerance. Buffering with back-off delivers every generated reading at an energy -14.9 to $+25.9\%$ different from that of not recovering at all on LoRaWAN, and hourly batching makes it cheaper still. A design that batches hourly, sends alarms immediately, and leaves early whenever the server's estimate would leave tolerance matched per-reading reporting on alarm latency and on the real-time picture on a held-out year, while per-reading ARQ

used 1.34×–2.53× as much transmission energy and 1.15×–1.41× as much node energy. A better probing schedule would save little, and once readings are batched, transmission is at most approximately half of the node's energy. What remains is the delivery gap itself, which can only be shortened by network engineering. Future work includes hardware measurements, longer delivery records from tropical deployments, the energy of NB-IoT cell search without a network, and recovery across several gateways.

Data and Code Availability

The USGS [60], InWaterSense [49], LoED [51], and UVA [50] data are public. The Weija records were provided by the deployment's operators [2]; their raw values are not redistributed, but the delivery trace derived from them (the time of every record of every stream, and whether it counts as a genuine delivery) is. The code, configurations, seeds, environment lock files, data-acquisition scripts, input checksums, the frozen confirmatory plan with the hashes of the frozen code, and the derived traces are archived at Zenodo [66] (doi:10.5281/zenodo.23455071). Every number in this paper is generated from the result files by a script that records its source.

Statements and Declarations

Funding: No funds, grants, or other support were received during the preparation of this manuscript.

References

[1] K. S. Adu-Manu, C. Tapparello, W. Heinzelman, F. A. Katsriku, and J.-D. Abdulai, "Water quality monitoring using wireless sensor networks: Current trends and future research directions," *ACM Transactions on Sensor Networks*, vol. 13, no. 1, pp. 1–41, 2017.

[2] K. S. Adu-Manu, F. A. Katsriku, J.-D. Abdulai, and F. Engmann, "Smart river monitoring using wireless sensor networks," *Wireless Communications and Mobile Computing*, vol. 2020, pp. 1–19, 2020, art. no. 8897126.

[3] F. Ahmedi and L. Ahmedi, "Dataset on water quality monitoring from a wireless sensor network in a river in Kosovo," *Data in Brief*, vol. 44, p. 108486, 2022.

[4] T. Le Dinh, W. Hu, P. Sikka, P. Corke, L. Overs, and S. Brosnan, "Design and deployment of a remote robust sensor network: Experiences from an outdoor water quality monitoring network," in *Proc. 32nd IEEE Conf. Local Computer Networks (LCN)*. IEEE, 2007, pp. 799–806.

[5] K. Langendoen, A. Baggio, and O. Visser, "Murphy loves potatoes: Experiences from a pilot sensor network deployment in precision agriculture," in *Proc. 20th IEEE Int. Parallel and Distributed Processing Symp. (IPDPS)*. IEEE, 2006, p. 8 pp.

[6] F. Ingelrest, G. Barrenetxea, G. Schaefer, M. Vetterli, O. Couach, and M. Parlange, "SensorScope: Application-specific sensor network for environmental monitoring," *ACM Transactions on Sensor Networks*, vol. 6, no. 2, pp. 17:1–17:32, 2010.

[7] M. Brzozowski, M. Frohberg, and P. Langendoerfer, "Real-world deployments of sensor networks: Practical lessons for researchers," in *Wired/Wireless Internet Communications (WWIC 2018), LNCS vol. 11618*. Springer International Publishing, 2018, pp. 42–53.

[8] P. P. Puluckul and M. Weyn, "Experiences with sub-arctic sensor network deployment," in *Proc. IEEE Wireless Communications and Networking Conf. (WCNC)*. IEEE, 2025, pp. 1–6.

[9] M. Zennaro, E. Pietrosemoli, A. Soltobaev, T. Sultanov, and B. Moldobekov, "From mountains to data: low-cost weather stations in Kyrgyzstan's challenging terrain," *Frontiers in Communications and Networks*, vol. 6, 2025, art. no. 1505375.

[10] B. Sinopoli, L. Schenato, M. Franceschetti, K. Poolla, M. Jordan, and S. Sastry, "Kalman filtering with intermittent observations," *IEEE Transactions on Automatic Control*, vol. 49, no. 9, pp. 1453–1464, 2004.

[11] K. Huang, W. Liu, Y. Li, and B. Vucetic, "To retransmit or not: Real-time remote estimation in wireless networked control," in *Proc. IEEE Int. Conf. Communications (ICC)*. IEEE, 2019, pp. 1–7.

[12] K. Huang, W. Liu, M. Shirvanimoghaddam, Y. Li, and B. Vucetic, "Real-time remote estimation with hybrid ARQ in wireless networked control," *IEEE Transactions on Wireless Communications*, vol. 19, no. 5, pp. 3490–3504, 2020.

[13] L. Kong, M. Xia, X.-Y. Liu, G. Chen, Y. Gu, M.-Y. Wu, and X. Liu, "Data loss and reconstruction in wireless sensor networks," *IEEE Transactions on Parallel and Distributed Systems*, vol. 25, no. 11, pp. 2818–2828, 2014.

[14] A. Cini, I. Marisca, and C. Alippi, "Filling the G_ap_s: Multivariate time series imputation by graph neural networks," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2022, arXiv:2108.00298.

[15] I. Marisca, A. Cini, and C. Alippi, "Learning to reconstruct missing data from spatiotemporal graphs with sparse observations," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022.

[16] Q. Chang, D. Tao, J. Wang, and R. Gao, "Deep compressed sensing based data imputation for urban environmental monitoring," *ACM Transactions on Sensor Networks*, vol. 20, no. 1, pp. 1–21, 2023, art. no. 17.

[17] J. Zhao and R. Govindan, "Understanding packet delivery performance in dense wireless sensor networks," in *Proc. 1st Int. Conf. Embedded Networked Sensor Systems (SenSys)*. ACM, 2003, pp. 1–13.

[18] L. Bhatia, M. Breza, R. Marfievici, and J. A. McCann, "LoED: The LoRaWAN at the edge dataset," in *Proc. 3rd Workshop on Data: Acquisition To Analysis (DATA '20)*. ACM, 2020, pp. 7–8.

[19] F. Nikseresht, V. A. L. Sobral, J. L. Goodall, and B. Campbell, "Dataset: Long-term LoRaWAN communication metadata from an urban deployment," in *Proc. 12th ACM Int. Conf. Systems for Energy-Efficient Buildings, Cities, and Transportation (BuildSys '25)*. ACM, 2025, pp. 412–417.

[20] R. D. Yates, Y. Sun, D. R. Brown, S. K. Kaul, E. Modiano, and S. Ulukus, "Age of information: An introduction and survey," *IEEE Journal on Selected Areas in Communications*, vol. 39, no. 5, pp. 1183–1210, 2021.

[21] B. Sun and N. Pappas, "Resource-constrained semantic-aware remote estimation with overlapping sensor coverage," in *Proc. ACM MobiHoc (to appear); arXiv:2609.07563*, 2026, to appear; arXiv:2609.07563.

[22] P. J. Marcelis, V. Rao, and R. V. Prasad, "DaRe: Data recovery through application layer coding for LoRaWAN," in *Proc. 2nd Int. Conf. Internet-of-Things Design and Implementation (IoTDI)*. ACM, 2017, pp. 97–108.

[23] M. Capuzzo, D. Magrin, and A. Zanella, "Confirmed traffic in LoRaWAN: Pitfalls and countermeasures," in *Proc. 17th Annu. Mediterranean Ad Hoc Networking Workshop (Med-Hoc-Net)*. IEEE, 2018, pp. 1–7.

[24] H.-N. Hoang, K. Nguyen, H. Sekiya, and J.-D. Kim, "Unified spatial analytical framework for confirmed uplinks in multi-gateway LoRaWAN under duty-cycle constraints," *IEEE Open Journal of the Communications Society*, vol. 7, pp. 1901–1922, 2026.

[25] J. Lee, Y. S. Yoon, H. W. Oh, and K. R. Park, "DG-LoRa: Deterministic group acknowledgment transmissions in LoRa networks for industrial IoT applications," *Sensors*, vol. 21, no. 4, 2021, art. no. 1444.

[26] K. Q. Abdelfadeel, D. Zorbas, V. Cionca, and D. Pesch, "FREE—fine-grained scheduling for reliable and energy-efficient data collection in LoRaWAN," *IEEE Internet of Things Journal*, vol. 7, no. 1, pp. 669–683, 2020.

[27] A. Jain, M. A. Haque, A. Saifullah, and H. Zhang, "Burst-MAC: A MAC protocol for handling burst traffic in LoRa network," in *Proc. IEEE Real-Time Systems Symp. (RTSS)*. IEEE, 2024, pp. 148–160.

[28] L. Acosta-Garcia, J. Aznar-Poveda, S. Pedratscher, M. Etheredge, R. Asorey-Cacheda, J. Garcia-Haro, T. Fahringer, and A.-J. Garcia-Sanchez, "CADD: Collaborative adaptive duty cycle and data reduction for congestion control in LoRaWAN," *Internet of Things*, vol. 37, 2026, art. no. 101904.

[29] C. Luo, F. Wu, J. Sun, and C. W. Chen, "Compressive data gathering for large-scale wireless sensor networks," in *Proc. 15th Annu. Int. Conf. Mobile Computing and Networking (MobiCom)*. ACM, 2009, pp. 145–156.

[30] J. Cheng, Q. Ye, H. Jiang, D. Wang, and C. Wang, "STCDG: An efficient data gathering algorithm based on matrix completion for wireless sensor networks," *IEEE Transactions on Wireless Communications*, vol. 12, no. 2, pp. 850–861, 2013.

[31] X. Piao, Y. Hu, Y. Sun, B. Yin, and J. Gao, "Correlated spatio-temporal data collection in wireless sensor networks based on low rank matrix approximation and optimized node sampling," *Sensors*, vol. 14, no. 12, pp. 23 137–23 158, 2014.

[32] D. Cao, Y. Wang, J. Duan, C. Zhang, X. Zhu, C. Huang, Y. Tong, B. Xu, J. Bai, J. Tong, and Q. Zhang, "Spectral temporal graph neural network for multivariate time-series forecasting," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 17 766–17 778.

[33] B. Jing, D. Zhou, K. Ren, and C. Yang, "Causality-aware spatiotemporal graph neural networks for spatiotemporal time series imputation," in *Proc. 33rd ACM Int. Conf. Information and Knowledge Management (CIKM)*. ACM, 2024, pp. 1027–1037.

[34] W. Du, D. Côté, and Y. Liu, "SAITS: Self-attention-based imputation for time series," *Expert Systems with Applications*, vol. 219, p. 119619, 2023.

[35] T. Nie, G. Qin, W. Ma, Y. Mei, and J. Sun, "ImputeFormer: Low rankness-induced transformers for generalizable spatiotemporal imputation," in *Proc. 30th ACM SIGKDD Conf. Knowledge Discovery and Data Mining (KDD)*. ACM, 2024, pp. 2260–2271.

[36] E. E. Peterson and J. M. Ver Hoef, "A mixed-model moving-average approach to geostatistical modeling in stream networks," *Ecology*, vol. 91, no. 3, pp. 644–651, 2010.

[37] P. McDonald, D. Geraghty, I. Humphreys, S. Farrell, and V. Cahill, "Sensor network with delay tolerance (SeNDT)," in *2007 16th International Conference on Computer Communications and Networks*. IEEE, 2007, pp. 1333–1338.

[38] M. J. Murphy, Ø. Tveito, E. F. Kleiven, I. Rais, E. M. Soininen, J. M. Bjørndalen, and O. Anshus, "Experiences building and deploying wireless sensor nodes for the arctic tundra," in *Proc. IEEE/ACM 21st Int. Symp. Cluster, Cloud and Internet Computing (CCGrid)*. IEEE, 2021, pp. 376–385.

[39] L. Casals, B. Mir, R. Vidal, and C. Gomez, "Modeling the energy performance of LoRaWAN," *Sensors*, vol. 17, no. 10, p. 2364, 2017.

[40] F. Michelinakis, A. S. Al-Selwi, M. Capuzzo, A. Zanella, K. Mahmood, and A. Elmokashfi, "Dissecting energy consumption of NB-IoT devices empirically," *IEEE Internet of Things Journal*, vol. 8, no. 2, pp. 1224–1242, 2021.

[41] A. Sørensen, H. Wang, M. J. Remy, N. Kjettrup, R. B. Sørensen, J. J. Nielsen, P. Popovski, and G. C. Madueño, "Modeling and experimental validation for battery lifetime estimation in NB-IoT and LTE-M," *IEEE Internet of Things Journal*, vol. 9, no. 12, pp. 9804–9819, 2022.

[42] A. Azari, C. Stefanovic, P. Popovski, and C. Cavdar, "On the latency-energy performance of NB-IoT systems in providing wide-area IoT connectivity," *IEEE Transactions on Green Communications and Networking*, vol. 4, no. 1, pp. 57–68, 2020.

[43] S. Ugwuanyi, G. Paul, and J. Irvine, "Survey of IoT for developing countries: Performance analysis of LoRaWAN and cellular NB-IoT networks," *Electronics*, vol. 10, no. 18, p. 2224, 2021, art. no. 2224.

[44] G. M. Dias, B. Bellalta, and S. Oechsner, "A survey about prediction-based data reduction in wireless sensor networks," *ACM Computing Surveys*, vol. 49, no. 3, pp. 1–35, 2016.

[45] S. Santini and K. Römer, "An adaptive strategy for quality-based data reduction in wireless sensor networks," in *Proc. 3rd Int. Conf. Networked Sensing Systems (INSS)*, 2006.

[46] D. Arnaiz, F. Moll, E. Alarcón, and X. Vilajosana, "Energy and relevance-aware adaptive monitoring method for wireless sensor nodes with hard energy constraints," *Integration*, vol. 94, p. 102097, 2024.

[47] I. R. Martínez-Sánchez, J. Cuellar-Padilla, J. Olivares, J. M. Palomares, and F. León-García, "Context-aware adaptive Send-on-Delta for traffic saving in sensor networks," *Ad Hoc Networks*, vol. 178, 2025, art. no. 103967.

[48] A. Silberstein, G. Puggioni, A. Gelfand, K. Munagala, and J. Yang, "Suppression and failures in sensor networks: A Bayesian approach," in *Proc. 33rd Int. Conf. Very Large Data Bases (VLDB)*, 2007, pp. 842–853.

[49] L. Ahmedi, "InWaterSense dataset: Data from a wireless sensor network on water quality monitoring in a river in Kosovo," Mendeley Data, V1, 2021. [Online]. Available: https://doi.org/10.17632/krzv3g6d5f.1

[50] F. Nikseresht, V. A. L. Sobral, J. L. Goodall, and B. Campbell, "Long-term LoRaWAN communication metadata from an urban deployment," University of Virginia Dataverse (LibraData), 2025. [Online]. Available: https://doi.org/10.18130/V3/RFTICK

[51] L. Bhatia, M. Breza, R. Marfievici, and J. A. McCann, "LoED: The LoRaWAN at the edge dataset," Zenodo, 2020. [Online]. Available: https://doi.org/10.5281/zenodo.4121430

[52] B. Efron, "Bootstrap methods: Another look at the jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.

[53] H. Akaike, "A new look at the statistical model identification," *IEEE Transactions on Automatic Control*, vol. 19, no. 6, pp. 716–723, 1974.

[54] Libelium Comunicaciones Distribuidas, *Waspmote Technical Guide (product line v15)*, Libelium Comunicaciones Distribuidas, 2026, online guide, Sec. 1.26 (power modes: on 17 mA, deep sleep 33 μA); accessed Nov. 2026.

[55] ——, *Smart Water Sensor Guide (product line v15)*, Libelium Comunicaciones Distribuidas, 2026, online guide, Sec. 1.18 (tables of consumption); accessed Nov. 2026.

[56] LoRa Alliance Technical Committee, "LoRaWAN specification v1.0.2," LoRa Alliance, Tech. Rep., 2016, sec. 18.4, Data-rate adaptation during message retransmissions.

[57] ——, "LoRaWAN L2 1.0.4 specification (TS001-1.0.4)," LoRa Alliance, Tech. Rep., 2020, sec. 4.3.1.3, Retransmission procedure; Sec. 5.3, LinkADRReq (NbTrans).

[58] LoRa Alliance Technical Committee Regional Parameters Workgroup, "RP002-1.0.3 LoRaWAN regional parameters," LoRa Alliance, Tech. Rep., 2021.

[59] R. J. Wagner, R. W. Boulger, C. J. Oblinger, and B. A. Smith, "Guidelines and standard procedures for continuous water-quality monitors: Station operation, record computation, and data reporting," U.S. Geological Survey, Reston, VA, USA, Techniques and Methods 1-D3, 2006.

[60] U.S. Geological Survey, "USGS water data for the nation (National Water Information System)," 2024, instantaneous values for 2023, accessed Oct. 2026. [Online]. Available: https://doi.org/10.5066/F7P55KJN

[61] B. B. Benson and D. Krause, "The concentration and isotopic fractionation of oxygen dissolved in freshwater and seawater in equilibrium with the atmosphere," *Limnology and Oceanography*, vol. 29, no. 3, pp. 620–632, 1984.

[62] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, p. 80, 1945.

[63] Winbond Electronics, *W25Q128JV 3V 128M-bit Serial Flash Memory, Datasheet, Rev. F*, Winbond Electronics, 2018, sec. 9.4 (DC characteristics) and AC electrical characteristics.

[64] R. E. Barlow, L. C. Hunter, and F. Proschan, "Optimum checking procedures," *Journal of the Society for Industrial and Applied Mathematics*, vol. 11, no. 4, pp. 1078–1095, 1963.

[65] E. S. Page, "Continuous inspection schemes," *Biometrika*, vol. 41, no. 1–2, pp. 100–115, 1954.

[66] S. Azongo, "river-telemetry-loss-handling, version v1.0.0: code, configurations and derived data," Zenodo, software, 2026. [Online]. Available: https://doi.org/10.5281/zenodo.23455071